Turning text into numbers

NLP, part 1. Everything a model does with text starts by counting something.

60 min | Recorded | Class 4 of 15

4

What this session covers

01

Why text is hard
2 slides

02

From text to tokens
2 slides

03

Counting words
3 slides

04

Words as positions
2 slides

05

Code walkthrough
5 slides

© 2026 Devansh Parapalli

Section 1 of 5

Why text is hard

Three problems every method tries to solve.

01

Text is hard for three reasons, and each method handles them differently

© 2026 Devansh Parapalli

Computers don’t read. **They count.**

Every method in this class and the next is a better answer to the question “which numbers?”

© 2026 Devansh Parapalli

Text is hard for three reasons, and each method handles them differently

Same word, different meaning	“The close is down to two accounts” versus “Ledgerline is down”. One is progress, one is an outage.
Different words, same meaning	“Invoices not showing”, “import stuck”, “has the batch finished”. Three phrasings, one queue.
Order changes meaning	“Reopen failed after receipt” versus “receipt failed after reopen”. Same words, different fix.

© 2026 Devansh Parapalli

Section 2 of 5

From text to tokens

Cleaning, splitting, and pulling out the things with a fixed shape.

01

Cleaning a ticket comes before any counting

02

Regular expressions handle anything with a fixed shape, and they never guess

© 2026 Devansh Parapalli

Cleaning a ticket comes before any counting

1

Strip signatures and quoted replies

2

Normalise case where it doesn’t carry meaning

3

Split into tokens: words or word pieces

4

Replace IDs and amounts with placeholders

5

Keep a copy of the original text

© 2026 Devansh Parapalli

Regular expressions handle anything with a fixed shape, and they never guess

LOL's IDs all follow patterns: tickets, purchase orders, InvoiceHub batches, contractor IDs, error codes. A regex finds them exactly, every time, for free.

Use regex first. Only reach for a model when the shape isn't fixed.

Ticket · `TKT-204117`

Purchase order · `PO-4500318822`

InvoiceHub batch · `IH-260301-07`

Contractor · `CW-48213`

© 2026 Devansh Parapalli

Section 3 of 5

Counting words

Bag of words, TF-IDF and similarity.

01

A bag of words counts each word and forgets the order

02

TF-IDF gives rare words more weight, so “enrolment” counts and “the” doesn’t

03

Similar tickets point in similar directions, and cosine similarity measures the angle

© 2026 Devansh Parapalli

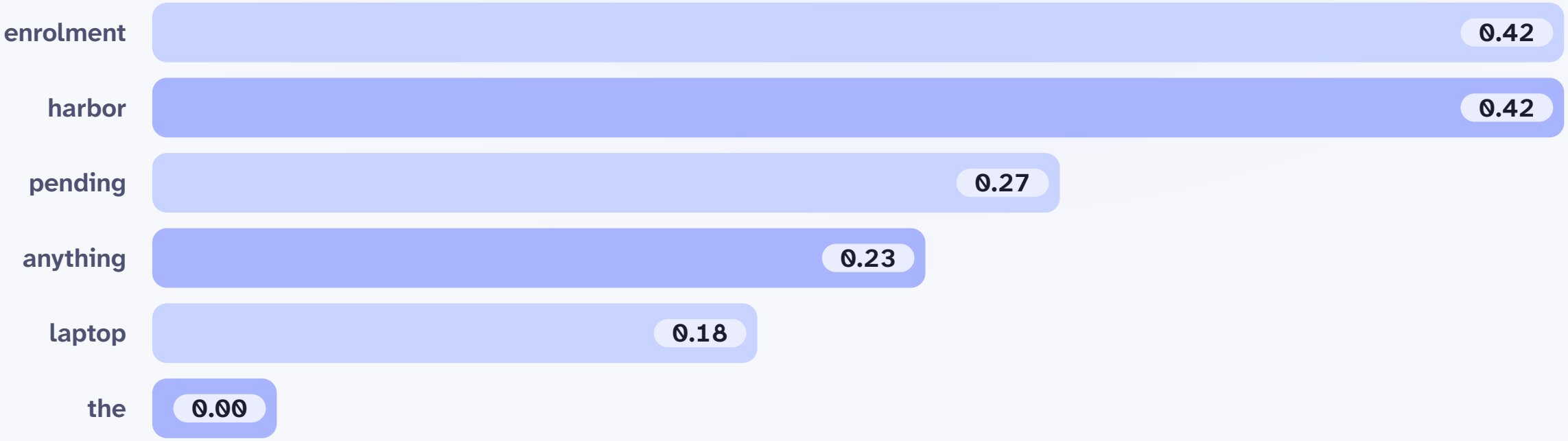
A bag of words counts each word and forgets the order

TICKET	IMPORT	INVOICE	REOPEN	FAILED
A · “Invoice import from ASS stuck, import shows error”	2	1	0	0
B · “Reopen failed after the invoice posted”	0	1	1	1
C · “Receipt failed after reopen”	0	0	1	1

© 2026 Devansh Parapalli

TF-IDF gives rare words more weight, so “enrolment” counts and “the” doesn’t

“Harbor MDM enrolment stuck. Laptop for CW-48213 shows enrolment pending in Harbor for two days. Can’t access anything until it completes.”



Source: Weights for one ticket, measured on the synthetic set

© 2026 Devansh Parapalli

Similar tickets point in similar directions, and cosine similarity measures the angle

New ticket: “Invoices from Armadillo Security Services not showing in InvoiceHub. Batch IH-260301-07.”

Once each ticket is a list of weights, two tickets can be compared by the angle between them. Note the PO ticket scoring 0.19 only because it names the same supplier.

a past “not showing” ticket	0.61
a status check, other wording	0.41
a PO reopen, same supplier	0.19
a GL reconciliation	0.01

© 2026 Devansh Parapalli

Section 4 of 5

Words as positions

Word vectors, and where each method falls over.

01

Word vectors put words with similar meanings close together

02

Each method fixes one problem and leaves another

© 2026 Devansh Parapalli

Word vectors put words with similar meanings close together

Train on enough text and every word gets a position in a few hundred dimensions. Words used in the same places end up near each other.

Now “import stuck” and “invoices not showing” can match even with no words in common.

import · upload · load · batch · feed

reopen · unclose · reactivate · amend

© 2026 Devansh Parapalli

Each method fixes one problem and leaves another

Regex	Exact and free. Misses anything not in the pattern.
Bag of words	Simple and fast. Ignores order and treats synonyms as unrelated.
TF-IDF	Weighs what matters. Still no order, still no synonyms, and fooled by shared names.
Word vectors	Understands synonyms. One meaning per word, whatever the sentence.

© 2026 Devansh Parapalli

Section 5 of 5

Code walkthrough

Pull IDs out of tickets, then find duplicates.

01

Seven patterns pull out most of what a ticket needs

02

TF-IDF plus cosine similarity finds duplicates in a few lines

03

The cutoff is a precision and recall choice, the same one from class 3

04

Where today sits on the card

© 2026 Devansh Parapalli

Seven patterns pull out most of what a ticket needs

code/class04/extract.py

```
PATTERNS = {
    "ticket":  re.compile(r"\bTKT-\d{6}\b"),
    "shipment": re.compile(r"\bLOL-SH-\d{7}\b"),
    "po":      re.compile(r"\bPO-4500\d{6}\b"),
    "batch":   re.compile(r"\bIH-\d{6}-\d{2}\b"),
    "worker":  re.compile(r"\bCW-\d{5}\b"),
    "error":   re.compile(r"\bERR-[A-Z]{2}-\d{4}\b"),
    "account": re.compile(r"\b\d{4}-\d{2}\b"),
}
```


© 2026 Devansh Parapalli

TF-IDF plus cosine similarity finds duplicates in a few lines

code/class04/duplicates.py

```
vec = TfidfVectorizer(ngram_range=(1, 2), min_df=2).fit(df.text)
sims = cosine_similarity(vec.transform([NEW]), vec.transform(df.text))[0]
likely = df[sims > CUTOFF].assign(score=sims[sims > CUTOFF]).sort_values("score")
```


© 2026 Devansh Parapalli

The cutoff is a precision and recall choice, the same one from class 3

- **Raise it** and you get fewer suggestions, nearly all real duplicates
- **Lower it** and you catch more, with more noise
- Pick it on a labelled sample, never by eye

0 matches	0.8
2 matches	0.6
9 matches	0.4

© 2026 Devansh Parapalli

Where today sits on the card

1. A script	The steps are known and don’t change.
2. A rule	**The decision is known and must be explainable line by line.**
3. Classical ML	**Labelled history, and the input is a table or short text.**
4. A small trained model	Lots of text, images or audio, enough labels, cost per call matters.
5. An LLM with a prompt	Varied language in and out, few labels, someone checks the output.
6. An LLM plus retrieval, tools or agents	Needs private knowledge, has to act, or takes several steps.
7. Not yet	No ground truth, needs a guarantee, or nobody owns the result.

© 2026 Devansh Parapalli

What to do next

1. Run `code/class04/extract.py` and add a pattern for invoice numbers.
2. Try three cutoffs in `duplicates.py` and note what changes.
3. Read the class 5 pre-reading on attention.

Questions after class: the course channel.

© 2026 Devansh Parapalli